

Atelier 3 : les statistiques du registre

Par **Robert DIASSÉ**, Ingénieur Logiciel et Data

> **Cours** : Python avancé · **Chapitre** : 3, Structures de données avancées · **Version** : 2026-10-23

Avant de commencer

Ce que vous allez construire	Le programme `statistiques.py`, qui répond aux questions de la direction de Yonnee Express sur 2 000 livraisons : quels quartiers, quels livreurs, quelles durées, quels retards, quels revenus
Durée	🟢 Débutant : 4 h · 🔵 Standard : 3 h · 🟣 Expert : 2 h (sans les défis)
Prérequis	Avoir lu le chapitre 3 ; votre dépôt `yonnee-express`
Fichiers fournis	Le registre complet (`registre_complet.csv`, 2 000 livraisons valides, de janvier 2025 à octobre 2026, même format qu'à l'atelier 2)
Rendu	Votre dépôt GitHub `yonnee-express`, avec l'étiquette `atelier-3`

> 📍 **Point de départ** : votre dépôt de l'atelier 2. Placez `registre_complet.csv` dans le dossier `donnees`, et créez `statistiques.py`. Ce fichier est **propre** (déjà validé) : pas besoin de la fonction `convertir` de l'atelier 2.

Comment travailler

Mêmes règles que les ateliers précédents. **La règle de cet atelier** : pour chaque étape, choisissez l'outil le plus adapté du chapitre (`Counter`, `defaultdict`, ensemble, `deque`, tuple nommé, tri avec `key`), et dites lequel en commentaire. Le programme affiche ses résultats étape par étape, avec un titre `--- Étape N : ...`.

Étape 1 : charger le registre en tuples nommés (25 min)

🎯 **Objectif** : une liste de livraisons dont les champs ont un nom et un type.

🛠️ **A faire**

1. Un tuple nommé `Livraison` avec les 8 champs du fichier (`numero`, `date`, `livreur`, `quartier`, `poids`, `montant`, `frais`, `duree_min`).
2. `charger(chemin)` : lit le fichier avec `csv.DictReader` (séparateur `;`), convertit chaque champ (`int`, `float`, `date.fromisoformat`) et renvoie la liste des `Livraison`.
3. Une constante `SEUIL_RETARD = 60` (en minutes).

✅ **Vérification** : `2000 livraisons, du 2025-01-01 au 2026-10-22`.

💡 **A comprendre** : pourquoi un tuple nommé convient-il ici mieux qu'un dictionnaire ? (Pensez à la section 7 du chapitre.)

```
git add .
git commit -m "Chargement du registre complet en tuples nommés"
```

Étape 2 : les quartiers et les livreurs (15 min)

🛠️ **A faire** : le nombre de livraisons par quartier, du plus fréquent au moins fréquent, et le livreur le plus actif.

✅ **Vérification** : Yoff 422, Plateau 410, Médina 393, Parcelles 390, Pikine 385 ; livreur le plus actif : Fatou, 420 livraisons.

Étape 3 : les durées par quartier (25 min)

🛠️ **A faire** : pour chaque quartier, la durée moyenne et la **médiane** des livraisons (`from statistics import median`), les quartiers rangés de la médiane la plus courte à la plus longue.

✅ **Vérification**

Quartier	Moyenne	Médiane
Médina	35,0 min	35,0 min
Plateau	39,8 min	40,0 min
Yoff	44,6 min	45,0 min
Parcelles	50,8 min	51,0 min
Pikine	54,5 min	54,0 min

💡 **A comprendre** : quel est l'intérêt de la médiane par rapport à la moyenne ? Inventez une liste de 5 durées où elles sont très différentes.

Étape 4 : le tableau croisé livreur x quartier (35 min)

🎯 **Objectif** : un dictionnaire imbriqué, affiché comme un tableau.

🛠️ A faire

1. `croise` : pour chaque livreur, le nombre de livraisons par quartier. Indice : `defaultdict(Counter)`, puis `croise[l.livreur][l.quartier] += 1`.
2. Affichez-le en tableau aligné : une ligne par livreur (triés), une colonne par quartier (triés), avec des f-strings (`f"{texte:>10}"` aligne à droite sur 10 caractères).

▼ 💡 Indice : l'en-tête du tableau

```
noms_quartiers = sorted(quartiers)
print(f"{'':8}" + "".join(f"{q:>10}" for q in noms_quartiers))
```

`"".join(...)` colle les colonnes ; `f"{'':8}"` laisse 8 espaces pour la colonne des noms.

✅ **Vérification** : la ligne d'Ibou est `Ibou 68 99 85 82 83` (Médina, Parcelles, Pikine, Plateau, Yoff) ; celle de Coumba : `85 62 74 79 85`.

💡 **A comprendre** : `defaultdict(Counter)` crée quoi, exactement, pour un livreur nouveau ?

```
git add .
git commit -m "Statistiques : quartiers, durées, tableau croisé"
```

Étape 5 : les retards (30 min)

🛠️ A faire

1. Pour chaque livreur : le nombre de retards (durée de plus de `SEUIL_RETARD` minutes), le nombre total de livraisons et le taux, du taux le plus élevé au plus faible.
2. Le nombre de retards par quartier.

✅ Vérification

Livreur	Retards	Livraisons	Taux
Awa	51	398	12,8 %
Ibou	51	417	12,2 %
Fatou	44	420	10,5 %
Coumba	34	385	8,8 %

Livreur	Retards	Livraisons	Taux
Modou	29	380	7,6 %

Retards par quartier : Pikine 109, Parcelles 63, Yoff 30, Plateau 5, Médina 2.

💡 **A comprendre** : la direction veut sanctionner Awa, qui a « le plus de retards ». Avant de lui répondre, comparez avec les retards par quartier et le tableau croisé de l'étape 4. Les retards dépendent-ils surtout du livreur ou du quartier ? Quelle autre analyse faudrait-il faire pour être juste avec Awa ?

Étape 6 : les frais par mois (20 min)

🛠️ **A faire** : le total des frais encaissés chaque mois (clé `l.date.strftime("%Y-%m")`, valeurs additionnées dans un `defaultdict(int)`), le nombre de mois, le meilleur mois et le total général.

✅ **Vérification** : 22 mois ; meilleur mois : `2025-07`, 221 500 F ; total : 4 172 000 F.

💡 **A comprendre** : le mois d'octobre 2026 est-il comparable aux autres ? Pourquoi ?

Étape 7 : les dernières livraisons de chaque livreur (20 min)

🛠️ **A faire** : pour chaque livreur, les numéros de ses **3 dernières** livraisons dans l'ordre chronologique. Triez d'abord les livraisons par `(date, numero)`, puis remplissez un `defaultdict(lambda: deque(maxlen=3))`.

✅ **Vérification** : Awa `[829, 1972, 204]`, Modou `[1371, 663, 1496]`.

💡 **A comprendre** : pourquoi `defaultdict(deque(maxlen=3))` ne marcherait pas, alors que `defaultdict(lambda: deque(maxlen=3))` marche ? (Que doit recevoir `defaultdict` ?)

Étape 8 : les ensembles (20 min)

🛠️ **A faire**

1. Pour chaque livreur, les quartiers où il n'a **jamais** livré (différence d'ensembles), ou « tous les quartiers ».
2. Les numéros des colis lourds (plus de 5 kg) et des livraisons en retard, sous forme d'ensembles ; combien sont **à la fois** lourds et en retard ?

✅ **Vérification** : tous les livreurs ont livré dans tous les quartiers ; 1 155 colis lourds, 209 retards, 117 à la fois lourds et en retard.

💡 **A comprendre** : 117 sur 209 retards concernent un colis lourd, soit 56 %. Or 58 % des colis sont lourds (1 155 sur 2 000). Le poids explique-t-il les retards ?

Puis étiquetez :

```
git add .
git commit -m "Statistiques : retards, frais par mois, dernières livraisons"
git tag atelier-3
git push
git push origin atelier-3
```

🏆 Défis (facultatifs)

1. **Le retard à quartier égal.** Pour chaque couple (livreur, quartier), calculez le taux de retard. Awa est-elle plus lente que les autres **dans les mêmes quartiers** ?
2. **Le rapport en JSON.** Enregistrez toutes les statistiques dans `sorties/statistiques.json` (chapitre 2). Attention : un `Counter` et un `defaultdict` s'enregistrent-ils directement ?
3. **Le jour de la semaine.** Quel jour de la semaine a le plus de livraisons ? Indice : `l.date.weekday()` (0 pour lundi) et une liste de noms des jours.
4. 🟣 **Le temps de calcul.** Chronométrez (avec `time.perf_counter()`) le calcul des retards par livreur fait avec une boucle et deux `Counter`, puis avec une liste en compréhension par livreur (`sum(1 for l in livraisons if l.livreur == nom ...)`). Laquelle est la plus rapide, et pourquoi ?

Grille d'autoévaluation

Je sais…	Oui	Pas encore
choisir la structure adaptée à une question (compter, regrouper, croiser, garder les derniers)	☐	☐
utiliser `Counter` et `defaultdict`, y compris imbriqués	☐	☐
trier et chercher le maximum selon un critère	☐	☐
utiliser les opérations d'ensembles pour comparer des groupes	☐	☐
lire des statistiques avec prudence avant de conclure	☐	☐